intuitive

TECHNICAL SPECIFICATION DOCUMENT

Agent Dashboard (AI Garage Agent Dashboard)

Attribute	Value
Document Version	1.0
Date	21 October 2026
Author	Arya Kayastha
Status	Draft — for review
Reference	Companion to BRD-AGD-001

1. Purpose, Scope, and Status

This Technical Specification Document (TSD) details the architectural components, interface definitions, data schemas, verificatior operational deployment procedures for the AI Garage Agent Dashboard. While the Business Requirements Document (BRD-AGD· articulates the portfolio tracking capabilities, accountability models, and governance requirements, this document defines how thc capabilities are constructed, secured, executed, and operated.

1.1 A Warning the Reader Must Not Skip

The following operational constraints and implementation boundaries govern this architecture:

1. **Zero IT Permissions Footprint:** The application operates strictly without tenant-level IT involvement. There is no Microsoft En AD) application registration, no enterprise single sign-on (SSO), no dedicated Exchange mailbox, and no corporate DNS dele Authentication relies solely on local credentials secured via `scrypt`. Transactional emails are restricted to Azure Communic Services (ACS) on an Azure-managed domain, constrained by a subscription quota of approximately 5 emails per minute and
2. **Databricks Delta Transactional Limitations:** Databricks SQL serves as the system of record. Delta Lake tables execute queri decoupled compute warehouse and **do not enforce cross-statement transactions, unique constraints, or foreign key refe integrity**. Idempotency, conflict resolution, and relational consistency must be enforced deterministically within application-la using atomic `MERGE` statements and structured write flows.
3. **Paused Deployment Phase Alignment:** As of the baseline date, Phase 1 is live in production. Phase 2a is fully built and teste isolated schema `agent-dashboard-test`, but production deployment is temporarily paused. Resuming production release applying live Delta migrations `010` through `013`, configuring container secrets, executing the first account creation link, a Azure Resource Manager tags.

2. Requirement Traceability (BRD → TSD)

The functional requirements defined in BRD-AGD-001 map to the following technical subsystem specifications:

- **TR-1.x:** Registry & Metadata Subsystem
- **TR-2.x:** Identity, Access & Session Subsystem
- **TR-3.x:** Update Lifecycle, Review & Workflow Engine
- **TR-4.x:** Tracking, Temporal Metrics & Escalation Subsystem
- **TR-5.x:** FinOps Ingestion, Model Pricing & Alerting Subsystem
- **TR-6.x:** Portfolio Analytics & Narrative Generation Subsystem
- **TR-7.x:** Agent Machine Push, Heartbeat & Telemetry Subsystem (Phase 2b)

intuitive

BRD ID	Technical Requirement(s)	
BR-01	**TR-1.1:** CRUD API for agent entities; marketplace schema flags; relational table mappings in Delta Lake.	
BR-02	**TR-2.1:** Time-delimited token generation (`crypto.randomBytes(32)`): 48-hour account invitation tokens and 1-hour password reset tokens.	
BR-03	**TR-2.2:** Role-based access control (RBAC) middleware enforcing authorization (`Agent Manager`, `Contributor`, `FinOps`, `Executive`) at the Express route level.	
BR-04	**TR-2.3:** Server-side assignment isolation. Contributor requests enforce `WHERE agent_id IN (assignments)` in database queries; non-assigned agent IDs evaluate to HTTP `404 Not Found`.	
BR-05	**TR-3.1:** Staged workflow submission states (`pending_review`, `approved`, `changes_requested`, `rejected`). State transitions to core entity table restricted to review approval.	
BR-06	**TR-3.2:** Review execution engine supporting atomic approval, parameter modification on approval, change requests, and rejections. Enforce mandatory reviewer comments on rejection/revision.	
BR-07	**TR-3.3:** Bulk review controller executing batched atomic update transitions per approved submission record.	
BR-08	**TR-3.4:** Bypass review route for `Agent Manager`: direct-write transactional pathway with direct attribution logging.	
BR-09	**TR-4.1:** Temporal ETA calculation engine; mandatory change reason validation; immutable ETA history logging; slip badge derivation formula.	
BR-10	**TR-4.2:** Blocker state management: mandatory blocker categorization, unblocker attribution, and timestamping (`blocked_since`).	
BR-11	**TR-4.3:** Node.js background scheduler: hourly chron check evaluating `blocked_since >= 72h`; trigger ACS mailer and write immutable escalation record.	
BR-12	**TR-4.4:** Append-only JSON/Delta audit logging service capturing `before` and `after` payloads, actor ID, and preserved snapshot entity names.	
BR-13	**TR-3.5:** Daily bulk submission interface; application-level MERGE operations restricting records to one update per agent per calendar day.	
BR-14	**TR-4.5:** Time-series aggregation endpoints returning chronological progress history and state progression data.	
BR-15	**TR-5.1:** Ingestion endpoint accepting single and batch telemetry payloads; execution of asynchronous token-to-cost conversion formulas.	
BR-16	**TR-5.2:** Effective-dated price lookups matching `timestamp >= effective_start AND timestamp < effective_end`; alert generation for threshold breaches and missing rate cards.	
BR-17	**TR-5.3:** Threshold evaluation engine (global vs. agent-specific overrides); notification aggregation badge counter.	
BR-18	**TR-6.1:** Statistical portfolio aggregator calculating month-over-month (MoM) metrics, delta rates, and standard deviation anomaly flags.	
BR-19	**TR-2.4:** User administration endpoints; instant session revocation via `token_invalid_before` timestamp modification.	
BR-20	**TR-7.1:** Per-agent hashed API key authentication; 5-minute interval ingestion daemon; 15-minute cron-based dead-man switch for production agents.	

intuitive

BRD ID	Technical Requirement(s)	
BR-21	**TR-7.2:** Outage state machine enforcing strict two-email maximum budget (offline transition event, recovery event after 15 minutes continuous health).	
BR-22	**TR-7.3:** Client SDK documentation and static copy-paste code snippets in Python, Node.js, cURL, and Go.	
BR-23	**TR-6.2:** LLM executive summary generator leveraging `` `@anthropic-ai/sdk` `` with strict fallback to deterministic mathematical summary formatting.	

3. Architecture

3.1 Logical Architecture

The AI Garage Agent Dashboard runs as a single-origin containerized service deployed to Azure Container Apps. The frontend is Page Application (SPA) built with React 19, Vite, and Material-UI (MUI), served statically via the Express Node.js application proces Express API manages incoming HTTP traffic, routes requests through Zod validation schemas, executes authentication and author policies, coordinates domain workflows, and interfaces with the underlying Databricks SQL Warehouse via the `` `@databricks/sql ``

Telemetry push from external agent runtimes (Phase 2b) and user HTTP sessions share a single unified API gateway. All asynchron such as blocker escalation scans and dead-man heartbeat evaluations—run in-process via Node.js background timers. Outbound dispatch asynchronously through Azure Communication Services, and narrative reporting falls back to a deterministic template if t Anthropic API call is unavailable or unconfigured.

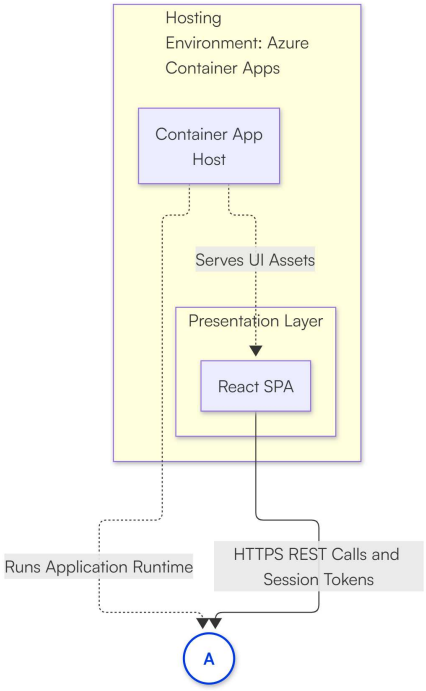


Figure 1 — Architecture (part 1 of 5)

intuitive

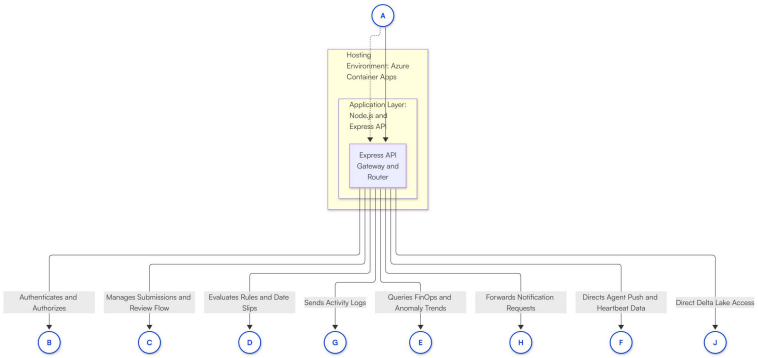


Figure 1 — Architecture (part 2 of 5)

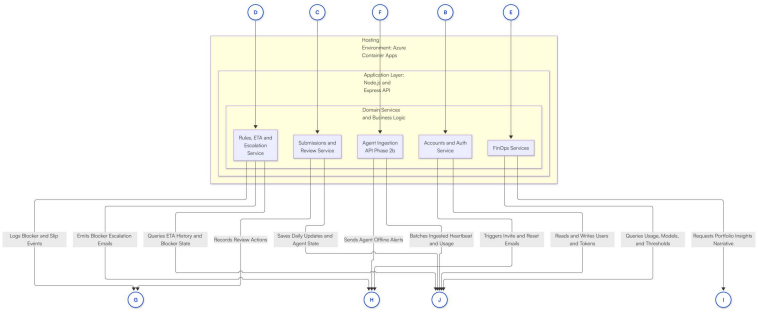


Figure 1 — Architecture (part 3 of 5)

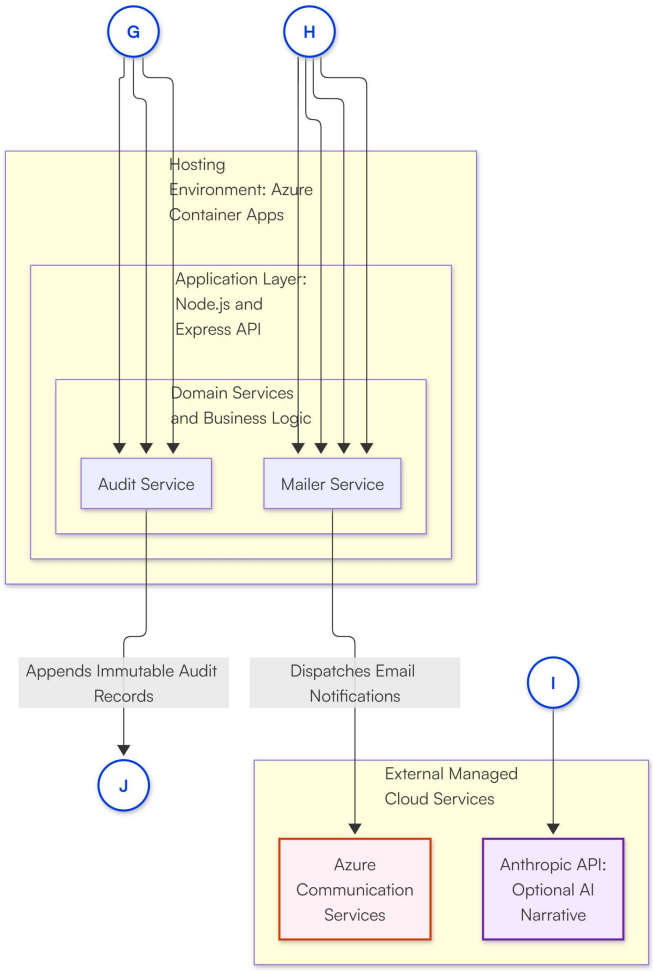


Figure 1 — Architecture (part 4 of 5)

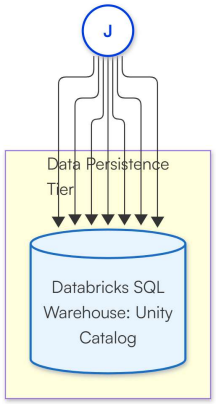


Figure 1 — Architecture (part 5 of 5)

3.2 Components

ID	Component	Responsibility	Technology
C-01	**React SPA**	Persona-based user interface, state management, client routing, data visualizations, and toast feedback.	React 19, Vite, MUI, TanStack Query Router, Recharts, react-hook-form
C-02	**Express API**	Gateway handling HTTP requests, cookie/token parsing, route dispatch, rate limiting, and Zod input validation.	Node.js 20, Express, plain JavaScri Zod

intuitive

ID	Component	Responsibility	Technology
C-03	**Accounts & Auth Service**	Credential hashing, invitation/reset workflows, account lockout tracking, and token session invalidation.	Node.js Crypto (`scrypt`), JSON V / Secure Tokens
C-04	**Submissions & Review Service**	Update submission persistence, review state machine transitions, reviewer modifications, and bulk approval batching.	Plain JavaScript, `@databricks/sq
C-05	**Rules, ETA & Escalation Service**	Slip badge calculation, blocker tracking, and hourly cron evaluating persistent blockers to send escalation emails.	Node.js background timers, JavaSc math
C-06	**Audit Service**	Append-only logging of mutations, state diffing (`before`/`after`), and denormalization of entity snapshots.	Plain JavaScript, Databricks Delta L
C-07	**FinOps Services**	Telemetry ingestion, effective-dated cost calculation, alert evaluation, and MoM statistical aggregations.	Plain JavaScript, Anthropic Node S
C-08	**Mailer**	Email compilation and queuing to Azure Communication Services under subscription rate limitations.	`@azure/communication-email`
C-09	**Agent Ingestion API (2b)**	API key validation, heartbeat ingestion, usage record ingestion, and offline detection logic.	Node.js 20, Express, Crypto (`timingSafeEqual`)
C-10	**Databricks Warehouse**	Primary system of record hosting relational Delta tables under Unity Catalog.	Databricks SQL Warehouse, Delta L
C-11	**Container App**	Host environment executing the single container image with unified static SPA and API server execution.	Azure Container Apps, Linux Alpine

3.3 The Load-Bearing Architectural Rule

> ***"All authorization scopes and entity access boundaries must be derived strictly on the server from the verified session ider client-provided actor IDs, workspace scopes, or agent ownership claims may be trusted.***
>
> *Code reviewers must reject any route or query where a client-supplied identifier filters records without joining against the authent server-resolved role and assignment mapping. If a user lacks assignment to a requested agent identifier, the system must return H* ***`Not Found`*** *rather than* ***`403 Forbidden`*** *to prevent resource enumeration."*

3.4 Data Flow

Step	From	To	Mechanism	Frequency
1	Client Web Browser	React SPA (C-01)	HTTPS load of static assets from Express host	Initial page l session start
2	React SPA (C-01)	Express API (C-02)	HTTPS POST `/api/auth/login`	On authenti
3	Express API (C-02)	Accounts Service (C-03)	Internal method invocation with `scrypt` verification	Per sign-in a
4	Express API (C-02)	Databricks (C-10)	SQL Query over ODBC/REST (`@databricks/sql`) to load user profile	Per sign-in a
5	Contributor User	Express API (C-02)	HTTPS POST `/api/updates/submit` with bearer token	Ad-hoc / Da
6	Express API (C-02)	Submissions Service (C-04)	Zod payload validation and assignment verification	Per update s
7	Submissions Service (C-04)	Databricks (C-10)	SQL `INSERT INTO agent_updates_pending`	Immediate c

intuitive

Step	From	To	Mechanism	Frequency
8	Agent Manager	Express API (C-02)	HTTPS POST `/api/reviews/:id/approve`	Within 2 bus
9	Submissions Service (C-04)	Databricks (C-10)	SQL `MERGE INTO agent_daily_updates` + `UPDATE agents`	Immediate c
10	Submissions Service (C-04)	Audit Service (C-06)	SQL `INSERT INTO audit_log` with diff payload	Immediate c
11	Escalation Service (C-05)	Databricks (C-10)	SQL query scanning open blockers > 72h	Scheduled h
12	Escalation Service (C-05)	Mailer (C-08)	Method call to queue escalation email	On blocker threshold
13	Mailer (C-08)	Azure Comm Services	HTTPS REST API call to ACS endpoint	Rate-limited 5/min)
14	External Agent Runtime	Agent Ingestion API (C-09)	HTTPS POST `/api/v1/telemetry/heartbeat` with API key	Every 5 min 2b)
15	Ingestion Daemon (C-09)	Databricks (C-10)	Batched SQL `INSERT INTO agent_heartbeats`	Batched eve seconds
16	FinOps Service (C-07)	Anthropic API	HTTPS POST `/v1/messages` (optional key)	Monthly / O

intuitive

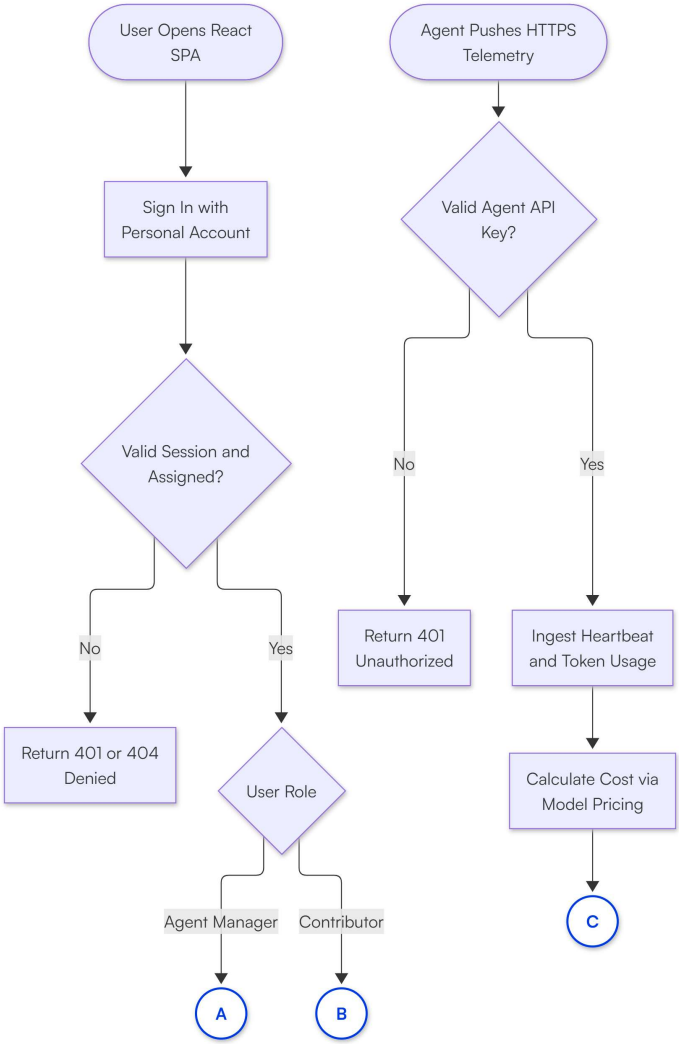


Figure 2 — Process Flow (part 1 of 4)

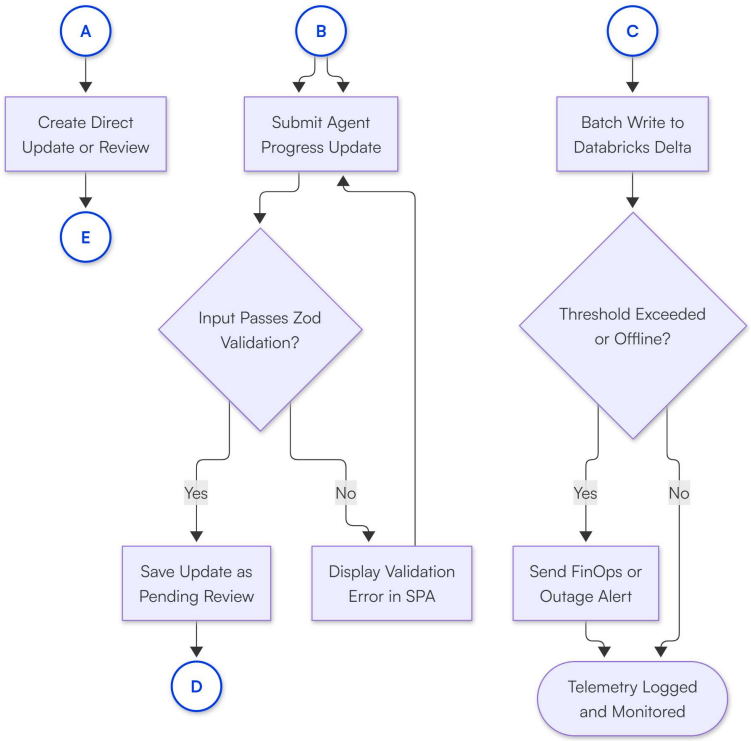


Figure 2 — Process Flow (part 2 of 4)

intuitive

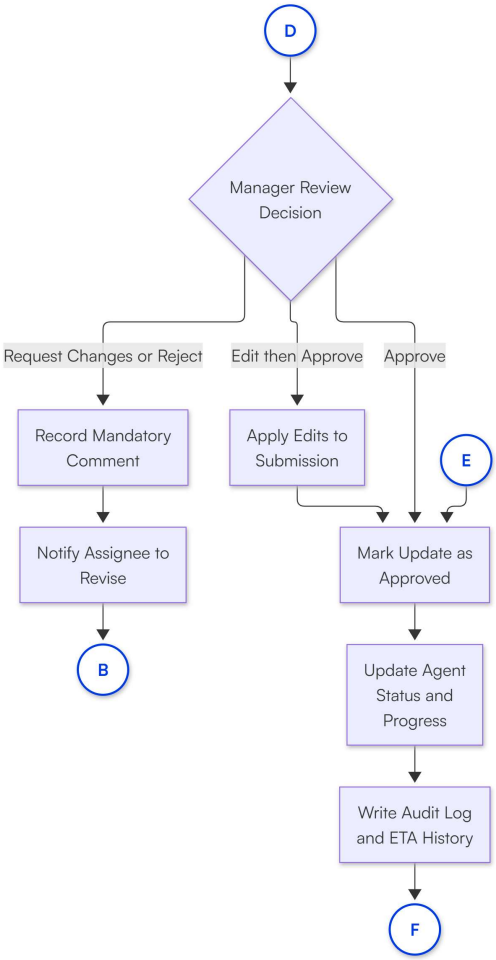


Figure 2 — Process Flow (part 3 of 4)

intuitive

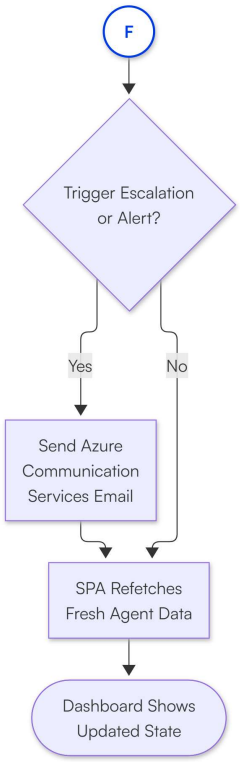


Figure 2 — Process Flow (part 4 of 4)

4. Environments

Environment	Databricks Catalog / Schema	Hosting Infrastructure	Purpose
Local Development	`hive_metastore.agent_dashboard_dev`	Node.js localhost:3000, Vite HMR	Developer implementation and
Test / Staging	`hive_metastore.agent-dashboard-test`	Azure Container Apps (`aca-agd-test`)	Pre-deployment schema valida integration test runs.
Production	`hive_metastore.agent_dashboard_prod`	Azure Container Apps (`aca-agd-prod`)	Live operational instance servir Garage contributors and leade

5. Source/System Integration

5.1 Databricks SQL Warehouse

- **Authentication Approach:** Personal Access Token (PAT) provisioned via service identity, passed as `DATABRICKS_TOKEN` env variable.
- **Access Method:** SQL execution via the `@databricks/sql` SDK over HTTPS (port 443).
- **Extraction Cadence:** Request-response transactional read/writes during user API operations; micro-batched (60-second wir for usage and heartbeat telemetry.
- **Rate / Resource Budget:** Shared SQL Warehouse compute. Operational queries use connection pooling and keep-alive cont prevent TCP socket resets.

intuitive

5.2 Azure Communication Services (ACS)

- **Authentication Approach:** Access key connection string stored in Container App Secret `ACS_CONNECTION_STRING`.
- **Access Method:** Azure Communication Email Client SDK (`@azure/communication-email`).
- **Rate / Cost Budget:** Strictly bounded to avoid triggering subscription blocks: maximum 5 emails/minute, 10 emails/hour. Cos at ~$0.00025 per message.
- **Object / Field Mapping:**

System Field	ACS API Field	Format / Handling
`sender_address`	`senderAddress`	Must match verified domain: `DoNotReply@<azure-managed-domain>.azurecomm.net`
`recipient_email`	`recipients.to[].address`	RFC 5322 formatted email string
`subject_line`	`content.subject`	UTF-8 plain string
`html_payload`	`content.html`	Clean standard HTML string; plain-text fallback populated

5.3 External Agents (Phase 2b Push)

- **Authentication Approach:** High-entropy Bearer token (`agd_live_` + 32-byte hex), hashed with SHA-256 for validation aga database records.
- **Access Method:** RESTful JSON push over TLS 1.3 to `/api/v1/telemetry/*`.
- **Extraction Cadence:** Push mechanism triggered by external agent schedulers (Heartbeats every 300 seconds; Usage pushe completion or batched hourly).

5.4 Anthropic API (Optional Integration)

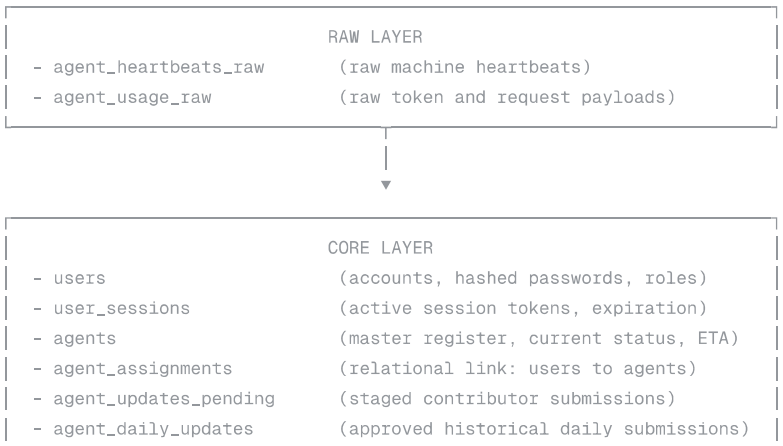
- **Authentication Approach:** `x-api-key` header mapped from Container App Secret `ANTHROPIC_API_KEY`.
- **Access Method:** Model call (`claude-3-5-sonnet-20241022` or current stable default) requesting JSON structured output f portfolio synthesis.
- **Cost Budget / Fallback:** Maximum 1 invocation per portfolio refresh; if API key is empty or call fails, system triggers hardcode deterministic synthesis engine.

---

6. Data Model

The data storage layer is hosted entirely in Databricks Delta Lake. Because Delta does not enforce primary keys or foreign keys, th layer guarantees integrity through write conventions and atomic merge operations.

6.1 Storage Layer Organization



intuitive

```
|  - agent_eta_history          (slip logs, old/new dates, reasons)      |
|  - agent_api_keys             (hashed machine keys for 2b push)        |
|  - finops_model_pricing       (effective-dated cost rate cards)        |
|  - audit_log                  (immutable event stream)                 |
└──────────────────────────────────────────────────────────────────────┘
                                  |
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
|                          AGGREGATE LAYER                             |
|  - finops_agent_monthly_agg   (monthly token count, cost, error rates) |
|  - portfolio_snapshots_weekly (historical executive metrics)          |
└──────────────────────────────────────────────────────────────────────┘
```

6.2 Relational Entities and Schemas

Core Entities: Users, Sessions, and Agents

```
CREATE TABLE users (
  id STRING NOT NULL,
  email STRING NOT NULL,
  password_hash STRING,
  full_name STRING NOT NULL,
  role STRING NOT NULL, -- 'Agent Manager', 'Contributor', 'Executive', 'FinOps'
  is_active BOOLEAN NOT NULL DEFAULT true,
  failed_login_attempts INT NOT NULL DEFAULT 0,
  locked_until TIMESTAMP,
  invite_token STRING,
  invite_expires_at TIMESTAMP,
  reset_token STRING,
  reset_expires_at TIMESTAMP,
  token_invalid_before TIMESTAMP,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL
) USING DELTA;

CREATE TABLE agents (
  id STRING NOT NULL,
  name STRING NOT NULL,
  description STRING,
  status STRING NOT NULL, -- 'Planned', 'In Development', 'Testing', 'In Production', 'Blocked', 'Retired'
  progress_pct INT NOT NULL DEFAULT 0,
  current_eta DATE,
  initial_eta DATE,
  is_marketplace BOOLEAN NOT NULL DEFAULT false,
  is_production BOOLEAN NOT NULL DEFAULT false,
  is_blocked BOOLEAN NOT NULL DEFAULT false,
  blocked_since TIMESTAMP,
  blocker_reason STRING,
  unblocker_assignee STRING,
  blocker_escalated BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL
) USING DELTA;

CREATE TABLE agent_assignments (
  agent_id STRING NOT NULL,
  user_id STRING NOT NULL,
```

intuitive

```
  created_at TIMESTAMP NOT NULL
) USING DELTA;
```

Update & Audit Subsystem

```
CREATE TABLE agent_updates_pending (
  id STRING NOT NULL,
  agent_id STRING NOT NULL,
  submitted_by_user_id STRING NOT NULL,
  status STRING NOT NULL, -- 'Pending', 'Approved', 'Changes Requested', 'Rejected'
  proposed_status STRING NOT NULL,
  proposed_progress_pct INT NOT NULL,
  proposed_eta DATE,
  summary_notes STRING,
  blocker_reason STRING,
  unblocker_assignee STRING,
  eta_change_reason STRING,
  reviewer_comments STRING,
  reviewed_by_user_id STRING,
  reviewed_at TIMESTAMP,
  created_at TIMESTAMP NOT NULL
) USING DELTA;

CREATE TABLE agent_daily_updates (
  agent_id STRING NOT NULL,
  update_date DATE NOT NULL,
  status STRING NOT NULL,
  progress_pct INT NOT NULL,
  summary_notes STRING,
  submitted_by_user_id STRING NOT NULL,
  approved_by_user_id STRING NOT NULL,
  created_at TIMESTAMP NOT NULL
) USING DELTA;

CREATE TABLE agent_eta_history (
  id STRING NOT NULL,
  agent_id STRING NOT NULL,
  previous_eta DATE,
  new_eta DATE NOT NULL,
  slip_days INT NOT NULL,
  change_reason STRING NOT NULL,
  changed_by_user_id STRING NOT NULL,
  created_at TIMESTAMP NOT NULL
) USING DELTA;

CREATE TABLE audit_log (
  id STRING NOT NULL,
  entity_type STRING NOT NULL, -- 'agent', 'user', 'pricing', 'review'
  entity_id STRING NOT NULL,
  entity_name_snapshot STRING NOT NULL,
  action STRING NOT NULL, -- 'CREATE', 'UPDATE', 'DELETE', 'APPROVE', 'ESCALATE'
  actor_user_id STRING NOT NULL,
  actor_name_snapshot STRING NOT NULL,
  before_payload_json STRING,
  after_payload_json STRING,
  created_at TIMESTAMP NOT NULL
) USING DELTA;
```

intuitive

FinOps & Machine Telemetry Entities

```
CREATE TABLE finops_model_pricing (
  id STRING NOT NULL,
  model_name STRING NOT NULL,
  provider STRING NOT NULL,
  input_token_cost_per_million DOUBLE NOT NULL,
  output_token_cost_per_million DOUBLE NOT NULL,
  effective_start_date DATE NOT NULL,
  effective_end_date DATE,
  created_at TIMESTAMP NOT NULL
) USING DELTA;

CREATE TABLE agent_usage_raw (
  id STRING NOT NULL,
  agent_id STRING NOT NULL,
  session_id STRING,
  model_name STRING NOT NULL,
  prompt_tokens INT NOT NULL,
  completion_tokens INT NOT NULL,
  total_cost DOUBLE,
  is_error BOOLEAN NOT NULL DEFAULT false,
  error_code STRING,
  duration_ms INT,
  recorded_at TIMESTAMP NOT NULL,
  ingested_at TIMESTAMP NOT NULL
) USING DELTA;

CREATE TABLE agent_api_keys (
  id STRING NOT NULL,
  agent_id STRING NOT NULL,
  key_prefix STRING NOT NULL,
  key_hash STRING NOT NULL,
  name STRING NOT NULL,
  is_revoked BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMP NOT NULL,
  last_used_at TIMESTAMP
) USING DELTA;

CREATE TABLE agent_heartbeats_raw (
  agent_id STRING NOT NULL,
  ip_address STRING,
  host_environment STRING,
  agent_version STRING,
  received_at TIMESTAMP NOT NULL
) USING DELTA;
```

6.3 Excluded by Construction

Under no circumstances does this architecture capture, process, or persist:

1. Prompts, completions, embeddings, or LLM payload context.
2. End-user business data, records, documents, or data processed by external agents.
3. Plaintext passwords or unhashed machine API keys.
4. Agent internal environmental variables, credentials, or container state dumps.

Any request payload received containing prompt text or conversational transcripts is discarded at the Zod ingestion validator bour database execution.

intuitive

7. Core Logic / Metric Implementation

7.1 Target Date (ETA) Slip Calculations

Every change to an agent's ETA where the proposed date is strictly later than the existing active ETA records a slip event.

```
/**
 * Calculates ETA slip count and total net slip days.
 * @param {Array<{previous_eta: string, new_eta: string}>} etaHistory
 * @param {string|null} initialEta
 * @param {string|null} currentEta
 * @returns {{slip_count: number, total_slip_days: number, badge_text: string|null}}
 */
function calculateEtaSlips(etaHistory, initialEta, currentEta) {
  if (!initialEta || !currentEta || etaHistory.length === 0) {
    return { slip_count: 0, total_slip_days: 0, badge_text: null };
  }

  let slipCount = 0;
  let totalSlipDays = 0;

  for (const record of etaHistory) {
    if (record.previous_eta && record.new_eta) {
      const prev = new Date(record.previous_eta).getTime();
      const next = new Date(record.new_eta).getTime();
      const diffDays = Math.round((next - prev) / (1000 * 60 * 60 * 24));

      if (diffDays > 0) {
        slipCount += 1;
        totalSlipDays += diffDays;
      }
    }
  }

  if (slipCount === 0) {
    return { slip_count: 0, total_slip_days: 0, badge_text: null };
  }

  return {
    slip_count: slipCount,
    total_slip_days: totalSlipDays,
    badge_text: `Slipped ${slipCount}× · +${totalSlipDays} days`
  };
}
```

Edge Case Handling: If **`initial_eta`** is null or equal to **`current_eta`**, slip count evaluates to 0. If an ETA is moved earlier (neg difference), the event is recorded in **`agent_eta_history`** with negative **`slip_days`**, but it does not increment **`slip_count`** o **`total_slip_days`**.

7.2 Hourly Blocker Escalation Engine

A background scheduler executes every 60 minutes on the application instance.

```
/**
 * Cron task: Escalate long-standing blockers.
 */
async function processBlockerEscalations(db, mailer, escalationThresholdDays = 3) {
```

intuitive

```
  const thresholdMs = escalationThresholdDays * 24 * 60 * 60 * 1000;
  const now = Date.now();

  const query = `
    SELECT id, name, blocked_since, blocker_reason, unblocker_assignee
    FROM agents
    WHERE is_blocked = true
      AND blocker_escalated = false
      AND blocked_since IS NOT NULL
  `;
  const blockedAgents = await db.query(query);

  for (const agent of blockedAgents) {
    const blockedDurationMs = now - new Date(agent.blocked_since).getTime();

    if (blockedDurationMs >= thresholdMs) {
      // 1. Fetch Agent Managers for escalation notification
      const managers = await db.query("SELECT email FROM users WHERE role = 'Agent Manager' AND is_active
      const recipientEmails = managers.map(m => m.email);

      // 2. Transmit notification via Mailer
      if (recipientEmails.length > 0) {
        await mailer.sendBlockerEscalationEmail({
          recipients: recipientEmails,
          agentName: agent.name,
          agentId: agent.id,
          blockedSince: agent.blocked_since,
          reason: agent.blocker_reason,
          unblocker: agent.unblocker_assignee,
          daysBlocked: Math.floor(blockedDurationMs / (1000 * 60 * 60 * 24))
        });
      }

      // 3. Mark agent as escalated to prevent duplicate emails
      await db.execute(`
        UPDATE agents
        SET blocker_escalated = true, updated_at = current_timestamp()
        WHERE id = '${agent.id}'
      `);

      // 4. Record escalation event in audit log
      await db.execute(`
        INSERT INTO audit_log VALUES (
          '${crypto.randomUUID()}', 'agent', '${agent.id}', '${agent.name.replace(/'/g, "''")}',
          'ESCALATE', 'system', 'Automated Escalation Service',
          NULL, '{"reason": "Blocker duration exceeded 3 days"}', current_timestamp()
        )
      `);
    }
  }
}
```

7.3 Effective-Dated FinOps Model Cost Calculation

Calculates the dollar cost of an inference invocation against historical pricing validity intervals.

```
/**
 * Derives dollar cost from model pricing tables.
```

intuitive

```
 */
function calculateCallCost(promptTokens, completionTokens, pricingRecords, callTimestamp) {
  const ts = new Date(callTimestamp).getTime();

  // Find applicable rate card based on date window
  const activePricing = pricingRecords.find(p => {
    const start = new Date(p.effective_start_date).getTime();
    const end = p.effective_end_date ? new Date(p.effective_end_date).getTime() : Infinity;
    return ts >= start && ts < end;
  });

  if (!activePricing) {
    // Unpriced model trigger: return null cost, raise alert flag
    return {
      cost: null,
      is_unpriced: true
    };
  }

  const promptCost = (promptTokens / 1_000_000) * activePricing.input_token_cost_per_million;
  const completionCost = (completionTokens / 1_000_000) * activePricing.output_token_cost_per_million;
  const totalCost = Number((promptCost + completionCost).toFixed(6));

  return {
    cost: totalCost,
    is_unpriced: false
  };
}
```

7.4 Portfolio Metrics and Anomaly Detection

```
/**
 * Calculates portfolio freshness, review velocity, and error anomalies.
 */
function calculatePortfolioMetrics(agents, reviews, monthlyUsage) {
  // Metric: Freshness (% active agents updated in last 7 days)
  const activeAgents = agents.filter(a => a.status !== 'Retired' && a.status !== 'Planned');
  const sevenDaysAgo = Date.now() - (7 * 24 * 60 * 60 * 1000);

  const freshCount = activeAgents.filter(a => {
    return a.last_approved_update && new Date(a.last_approved_update).getTime() >= sevenDaysAgo;
  }).length;

  const freshnessPct = activeAgents.length > 0
    ? Number(((freshCount / activeAgents.length) * 100).toFixed(1))
    : 0.0;

  // Metric: Review velocity (% updates reviewed within 2 business days)
  // Two business days approximated as 48 hours for calculation engine
  const twoDaysMs = 48 * 60 * 60 * 1000;
  const timelyReviews = reviews.filter(r => {
    const duration = new Date(r.reviewed_at).getTime() - new Date(r.created_at).getTime();
    return duration <= twoDaysMs;
  });

  const reviewVelocityPct = reviews.length > 0
    ? Number(((timelyReviews.length / reviews.length) * 100).toFixed(1))
    : null; // Null indicates no review volume rather than 0% compliance
```

intuitive

```
  return {
    freshnessPct,
    reviewVelocityPct,
    activeCount: activeAgents.length
  };
}
```

7.5 Agent Dead-Man Health State Engine (Phase 2b)

```
/**
 * Heartbeat evaluation state machine. Runs every 60 seconds.
 */
async function evaluateAgentLiveness(db, mailer) {
  const fifteenMinutesAgo = new Date(Date.now() - (15 * 60 * 1000)).toISOString();

  // Query production agents where last heartbeat is older than 15 minutes
  const offlineQuery = `
    SELECT a.id, a.name, MAX(h.received_at) as last_seen
    FROM agents a
    LEFT JOIN agent_heartbeats_raw h ON a.id = h.agent_id
    WHERE a.is_production = true AND a.health_status != 'Offline'
    GROUP BY a.id, a.name
    HAVING last_seen < '${fifteenMinutesAgo}' OR last_seen IS NULL
  `;
  const offlineAgents = await db.query(offlineQuery);

  for (const agent of offlineAgents) {
    // Transition to Offline, send Outage Email 1 of 2
    await db.execute(`UPDATE agents SET health_status = 'Offline' WHERE id = '${agent.id}'`);
    await mailer.sendOfflineNotification(agent.id, agent.name, agent.last_seen);
  }

  // Check recovery: Agents marked Offline that have maintained heartbeats for >= 15 continuous minutes
  const recoveryQuery = `
    SELECT a.id, a.name, MIN(h.received_at) as stable_since
    FROM agents a
    JOIN agent_heartbeats_raw h ON a.id = h.agent_id
    WHERE a.is_production = true
      AND a.health_status = 'Offline'
      AND h.received_at >= '${fifteenMinutesAgo}'
    GROUP BY a.id, a.name
    HAVING COUNT(*) >= 3 -- At least 3 successful pings in 15m window
  `;
  const recoveredAgents = await db.query(recoveryQuery);

  for (const agent of recoveredAgents) {
    // Transition to Healthy, send Recovery Email 2 of 2
    await db.execute(`UPDATE agents SET health_status = 'Healthy' WHERE id = '${agent.id}'`);
    await mailer.sendRecoveryNotification(agent.id, agent.name, agent.stable_since);
  }
}
```

intuitive

8. Interface Specifications

8.1 API Endpoint Matrix

Method	Path	Request Body	Returns	A F
`POST`	`/api/auth/login`	`{ email, password }`	`{ user, token }`	N
`POST`	`/api/auth/invite`	`{ email, role, fullName }`	`{ inviteUrl }`	A N
`POST`	`/api/auth/accept-invite`	`{ token, password }`	`{ status: "ok" }`	N
`POST`	`/api/auth/reset-request`	`{ email }`	`{ status: "ok" }`	N
`POST`	`/api/auth/reset-confirm`	`{ token, newPassword }`	`{ status: "ok" }`	N
`GET`	`/api/agents`	None	`{ data: Agent[] }`	C
`POST`	`/api/agents`	`{ name, description, eta, ... }`	`{ data: Agent }`	A N
`GET`	`/api/agents/:id`	None	`{ data: AgentDetail }`	A N
`PATCH`	`/api/agents/:id`	`{ name, description, isMarketplace }`	`{ data: Agent }`	A N
`POST`	`/api/updates/submit`	`{ agentId, status, progress, notes, blocker... }`	`{ submissionId }`	A C
`GET`	`/api/reviews/pending`	None	`{ data: PendingReview[] }`	A N
`POST`	`/api/reviews/:id/action`	`{ action: "approve"|"modify"|"request_changes"|"reject", modifications?, comments? }`	`{ status: "ok" }`	A N
`POST`	`/api/reviews/bulk-approve`	`{ submissionIds: string[] }`	`{ approvedCount: number }`	A N
`GET`	`/api/finops/overview`	`?month=YYYY-MM`	`{ summary, agentsUsage }`	C
`POST`	`/api/finops/pricing`	`{ modelName, provider, inputCost, outputCost, effectiveDate }`	`{ pricingId }`	F
`GET`	`/api/audit`	`?agentId=&actorId=&limit=50`	`{ data: AuditEntry[] }`	A N
`POST`	`/api/v1/telemetry/heartbeat`	`{ environment, version }`	`{ status: "received" }`	A K 2

intuitive

Method	Path	Request Body	Returns	A F
`POST`	`/api/v1/telemetry/usage`	`{ events: UsageRecord[] }`	`{ processed: number }`	A K 2

8.2 Standard Envelope Shape

All API payloads conform to the following JSON structure:

```
{
  "success": true,
  "data": {},
  "error": null,
  "timestamp": "2026-10-13T12:00:00.000Z"
}
```

Error envelopes return:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "INVALID_STATE_TRANSITION",
    "message": "Moving an ETA later requires a mandatory reason.",
    "details": []
  },
  "timestamp": "2026-10-13T12:00:00.000Z"
}
```

8.3 Interface Specifications and Constraints

1. **Authentication Enforcement:** User endpoints expect a cryptographically signed JSON Web Token passed in the HTTP `Aut Bearer <token>` header or via a secure `HttpOnly` cookie. Telemetry endpoints expect an `X-Agent-API-Key` header.
2. **Server-Side Request Scope Resolution:** Under no circumstances shall an API route accept an identity parameter (such as ` `userRole`) from client request bodies or query strings to partition data. The execution context resolves identity strictly via m inspecting the decrypted session token. For users possessing the `Contributor` role, all queries implicitly join against `agent_assignments`. If a Contributor attempts to access `/api/agents/:id` where `:id` is not assigned to them, the rou must return HTTP `404 Not Found`.
3. **Pagination Standard:** Listing routes (`/api/audit`, `/api/agents`) default to cursor-based limits: `?limit=50&cursor= <timestamp_or_id>`. Responses include `meta: { hasMore: boolean, nextCursor: string|null }`.
4. **Versioning Policy:** Web UI routes default to unversioned `/api/`. Phase 2b machine telemetry routes are strictly versioned u `/api/v1/`. Breaking changes to telemetry models require introducing `/api/v2/`.
5. **Input Validation:** Every mutating route validates payloads via Zod schemas prior to controller dispatch. Malformed payloads return HTTP `400 Bad Request` with field-level constraint errors.

---

9. Security and Privacy

9.1 Multi-Layer Defense in Depth

Data exclusion and authorization are enforced through independent layers:

1. **Database Query Scope Layer:** Contributor entity queries append `WHERE id IN (SELECT agent_id FROM agent_assignm user_id = :authenticated_user_id)`.

intuitive

2. **Controller Boundary Layer:** The Express route handlers verify ownership and role capabilities before initiating transactions.
3. **API Serialization Layer:** Data sanitization schemas explicitly strip internal fields, password hashes, and machine key secrets fr responses.
4. **Ingestion Exclusion Layer:** Prompt contents and enterprise raw data are completely absent from schemas and stripped by Z if present in raw HTTP payloads.

9.2 Authentication & Credential Storage

- **Password Storage:** User credentials are encrypted using Node.js `crypto.scrypt` with a cryptographically generated 16-by salt, an output key length of 64 bytes, and CPU/memory cost parameters $N=16384, r=8, p=1$.
- **Brute-Force Lockout:** Five consecutive failed authentication attempts against an email address lock the account for 15 minut (`locked_until = now() + 15m`).
- **Session Lifecycle:** Immediate token revocation occurs on password reset, account deactivation, or role modification by adva user's `token_invalid_before` timestamp. Requests bearing tokens issued prior to that timestamp fail validation.

9.3 Machine API Key Security (Phase 2b)

Machine keys follow the format: `agd_live_<8-character-key-id>_<32-character-secret>`.

- The server stores only the SHA-256 hash of the full key (`key_hash`) and displays the full plaintext key exactly once during ge
- Key verification uses `crypto.timingSafeEqual` over pre-computed SHA-256 hashes to prevent timing attack vulnerabilitie

10. Non-Functional Requirements

Category	Requirement Specification
Security	Passwords stored as `scrypt` hashes (12+ characters enforced). 5 failed sign-ins lock account for 15 minutes. Deactivation, role updates, and password resets terminate active sessions immediately. Machine access via per-agent hashed API keys. Rate limits enforced per route. Input payloads parsed through Zod validation. No credentials stored in source control.
Access Control	Server-side RBAC enforcement across all endpoints. Contributors isolated to assigned agents; unauthorized IDs yield HTTP 404.
Auditability	Append-only audit log capturing all mutations with before/after state diffs. Entity and user names preserved via snapshot fields against subsequent deletions.
Availability	Azure Container App scaled with min-replicas = 1 and max-replicas = 1 to preserve in-process dead-man detection state and avoid concurrency race conditions.
Performance	Dashboard baseline queries execute within < 3 seconds under typical network conditions. Machine usage and heartbeat telemetry ingested in memory and flushed via micro-batched writes to Databricks.
Data Integrity	Daily updates enforce a single entry per agent per calendar day via application-level `MERGE` logic. Partial failure execution paths fall back cleanly with documented error states.
Usability	Single-page application responsive across viewport widths from 375px (mobile) to 4K displays. Dark and Light MUI themes supported. Toast alerts displayed upon state persistence.
Testability	Unit tests and ESLint (`oxlint`) run within GitHub Actions. Integration test suites validate all migrations and business logic against `agent-dashboard-test` prior to release.
Cost	Operational footprint optimized for low monthly overhead. Transactional notification delivery bounded to ~$0.00025 per email on shared ACS resources.
Maintainability	Application structured into discrete architectural layers (`routes/`, `controllers/`, `services/`, `db/`). Code written in modular Node.js plain JavaScript (ESM) with comprehensive internal documentation.

intuitive

11. Testing

11.1 Test Levels & Tooling

- **Unit Tests:** Execute via `vitest`. Covers core utilities: `scrypt` hashing, token generation, ETA slip formulas, date interval r Zod schema validations.
- **Controller / Contract Tests:** Mock database and ACS SDK calls to assert correct HTTP response envelopes, status codes, ar authorization rejections.
- **Integration Tests:** Connect to the live Databricks SQL Warehouse targeting the isolated schema `agent-dashboard-test`. ` Delta migrations `001` through `013`, relational integrity emulations, `MERGE` statements, and transactional error recovery.

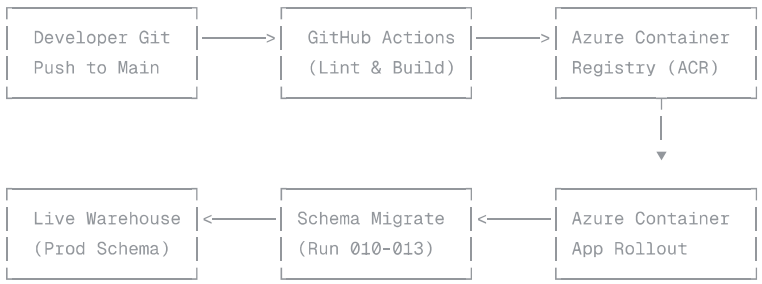
11.2 Release Gate Policy

> *"**RELEASE GATE: Pre-Deployment Integration Suite***
>
> *Prior to applying changes or rolling out container revisions to the live production environment, the entire integration test suite mus cleanly against the `agent-dashboard-test` Databricks schema from the deployment staging runner. Because Databricks crede stored in GitHub Actions, this suite runs locally or via an administrative deployment terminal. A single test failure, unresolved migra schema mismatch completely halts deployment."*

12. Deployment and Operations

12.1 Containerized Deployment Pipeline

The application packages into an OCI container image hosted in Azure Container Registry (ACR) and executed via Azure Containe (ACA).



1. **Continuous Integration:** GitHub Actions executes `oxlint`, validates React production bundle builds, and runs local unit te
2. **Container Build:** Docker multi-stage build creates the production Linux Alpine image containing compiled React frontend as Express backend services.
3. **Azure Policy Compliance:** Deployments must comply with subscription tags:
   - `Practice`: Specified corporate cost center tag.
   - `CreatedMonth`: Managed via deploy script update; appends the current deployment month to satisfy Azure tagging er policies.
4. **Secret Configuration:** Secrets injected directly into ACA environment variables:
   - `DATABRICKS_HOST`, `DATABRICKS_PATH`, `DATABRICKS_TOKEN`
   - `ACS_CONNECTION_STRING`, `EMAIL_SENDER`
   - `APP_URL`
   - `ANTHROPIC_API_KEY` (Optional)

intuitive

12.2 Observability and Operational Signals

Signal	Source	Condition	Alert Action
API Error Spike	Container App Log Analytics	> 5% requests returning HTTP 5xx over 5 min	Ops investigation; check Databricks warel
Databricks Reset	Express Server Logs	`ECONNRESET` or socket timeout from warehouse	Express retry mechanism triggers; logs ale consecutive failures occur.
Escalation Trigger	Background Cron Engine	Agent blocked duration $\ge$ 3 days	Dispatch email to Agent Managers; write a entry.
Agent Offline (2b)	Telemetry Ingestion Daemon	Production agent missing heartbeat for 15 min	Dispatch single outage notification to assig owners.
ACS Rate Bounding	Mailer Service	Send requests approach 5/min or 10/hr	Internal queue delays dispatch to prevent quota drops.

12.3 Disaster Recovery and Data Backups

Databricks Delta Lake natively tracks historical revisions through Delta Time Travel. In the event of a catastrophic corrupting write migration execution:

1. Revert table states using SQL: `RESTORE TABLE <table_name> TO TIMESTAMP AS OF <pre_incident_timestamp>`.
2. Application logs stream continuously to Azure Log Analytics Workspace, providing an external audit trail of requests and syste

13. Technical Risks

ID	Technical Risk	Impact	Mitigation Strategy
TR-RISK-01	**Databricks Connection Resets (`ECONNRESET`):** Queries to the Databricks SQL endpoint drop intermittently during compute scaling or network reset, causing slow (~25 min) integration test runs and transient API 500s.	Medium	Implement an exponential backoff retry wra the database query execution layer. Run ful suites prior to deployment releases rather tl during continuous build stages.
TR-RISK-02	**Non-Atomic Dual Writes:** Delta Lake does not support cross-statement transactions. Updating `agent_daily_updates` followed by updating the master `agents` record could fail midway, leaving current state stale relative to history.	Medium	Order operations deliberately: write the hist update log first, then apply the state update secondary write fails, the subsequent contr submission or review approval will self-heal record.
TR-RISK-03	**Single Reviewer Throughput Bottleneck:** Review workflow defaults solely to the Agent Manager role, creating review lag as portfolio size grows.	Medium	Design the authorization check around a `can_review` capability attribute rather tha hardcoded user identity, allowing future de team leads without schema refactoring.
TR-RISK-04	**ACS Notification Quota Exhaustion:** Sending emails over the shared Azure-managed domain risks rejection if volume exceeds ~5/minute or ~10/hour.	Medium	Enforce an in-memory leaky-bucket queue dispatch. Provide explicit "Copy Invite Link" directly in the user interface to eliminate ab dependence.
TR-RISK-05	**Immediate Migration Escalation Storm:** Applying Migration `013` backfills the `blocked_since` column for historical blocked agents, causing the hourly cron job to trigger emails for all long-blocked agents on initial rollout.	Medium	Pre-seed `blocker_escalated = true` d execution of Migration `013` for all agents blocked in production prior to activating th background scheduler.

intuitive

ID	Technical Risk	Impact	Mitigation Strategy
TR-RISK-06	**Azure Policy Tag Rejection:** Deployments fail ARM validation if the required `Practice` tag is missing or if the Azure append policy conflicts with an out-of-date `CreatedMonth` tag.	Medium	Script the ARM template deployment wrapp dynamically evaluate `CreatedMonth` at d runtime and supply required governance ta rollout.
TR-RISK-07	**Machine Telemetry Database Contention (Phase 2b):** High-frequency agent heartbeats pushing every 5 minutes create excessive file write operations and table locking in Delta Lake.	Medium	Buffer incoming heartbeats in an in-memor buffer within the Node.js process and flush batched single-statement `MERGE` once pe
TR-RISK-08	**Secret and Token Exposure:** Exposure of container secrets or machine API keys via API outputs or client-side bundles.	Medium	Maintain distinct Zod serialization output sc strip sensitive hash fields. Store all secrets i App Secret stores; never check credentials code.

14. Assumptions, Dependencies, and Open Questions

14.1 Architectural Assumptions

- Databricks compute warehouse is configured with auto-restart capabilities to resume processing when HTTP queries arrive.
- The application will run as a single container replica (`replicas: 1`), ensuring local in-process memory structures (such as c and cron intervals) operate reliably without distributed Redis caching.

14.2 Technical Dependencies

1. **Databricks SQL Warehouse:** Provisioned endpoint, valid catalog/schema, and a non-expiring Service Principal Personal Acc
2. **Azure Resources:** Azure Container App environment, Azure Container Registry, Log Analytics Workspace, and Azure Commu Services with a verified managed sender domain.
3. **Deployment Verification:** Local execution of integration test suites across migrations `001` through `013` against the `ager dashboard-test` schema prior to production cutover.
4. **Third-Party SDKs:** `@databricks/sql` version compatibility with Delta Lake Unity Catalog; `@azure/communication-email` `@anthropic-ai/sdk`.

14.3 Technical Open Questions

1. **Machine Agent Onboarding Protocol (Phase 2b):** Which repository structure and deployment model will distribute the clien Python/TypeScript agent builders?
2. **Data Retention & Pruning Lifecycle:** What is the automated table compaction and data retention schedule for `agent_hear` and `agent_usage_raw`? (Recommendation: Vacuum and purge raw pings after 90 days; retain monthly aggregations indef
3. **Production Definition Flag:** What exact programmatic criterion flags an agent as `is_production = true` to activate the 1 dead-man switch?
4. **Phase 3 Entra Transition Path:** When IT approvals are granted in Phase 3, what is the migration strategy to link local `scryp` records with corporate Azure AD object identifiers (`oid`)?

15. Appendix

15.1 Core Internal Domain Objects (JSON Schemas)

Contributor Update Submission Entity (`agent_updates_pending`)

```
{
  "id": "sub_8f9a2c1b-4d3e-4a5f-8b2c-1a3b5c7d9e0f",
  "agentId": "agt_01h7x9b2e4g6h8j1k3m5n7p9q1",
```

```
  "submittedByUserId": "usr_99e8d7c6-b5a4-3210-fedc-ba9876543210",
  "status": "Pending",
  "proposedStatus": "In Development",
  "proposedProgressPct": 65,
  "proposedEta": "2026-11-15",
  "summaryNotes": "Completed data pipeline extraction module; encountering schema drift on secondary inges
  "blockerReason": null,
  "unblockerAssignee": null,
  "etaChangeReason": "Additional testing cycles needed for edge cases.",
  "reviewerComments": null,
  "reviewedByUserId": null,
  "reviewedAt": null,
  "createdAt": "2026-10-13T09:30:00.000Z"
}
```


Historical ETA Slip Audit Object (`agent_eta_history`)


```
{
  "id": "slip_12345678-abcd-ef01-2345-6789abcdef01",
  "agentId": "agt_01h7x9b2e4g6h8j1k3m5n7p9q1",
  "previousEta": "2026-10-31",
  "newEta": "2026-11-15",
  "slipDays": 15,
  "changeReason": "Additional testing cycles needed for edge cases.",
  "changedByUserId": "usr_99e8d7c6-b5a4-3210-fedc-ba9876543210",
  "createdAt": "2026-10-13T14:22:10.000Z"
}
```


Telemetry Heartbeat Push Payload (`agent_heartbeats_raw`) (Phase 2b)


```
{
  "environment": "production-azure-eastus",
  "version": "1.4.2",
  "metadata": {
    "nodeVersion": "v20.11.0",
    "uptimeSeconds": 86400
  }
}
```


Telemetry Usage Record Payload (`agent_usage_raw`) (Phase 2b)


```
{
  "events": [
    {
      "sessionId": "sess_987654321",
      "modelName": "claude-3-5-sonnet-20241022",
      "promptTokens": 1420,
      "completionTokens": 380,
      "isError": false,
      "errorCode": null,
      "durationMs": 1240,
      "recordedAt": "2026-10-13T10:14:02.000Z"
    }
  ]
}
```

intuitive

16. Current Deployment Status

Phase 1 is live on Azure Container Apps. The Phase 2a deploy is paused partway through. Done: the Azure Communication Servic and sender domain, the email connection secret and the resource tags. Remaining: migrations 010—013 on the live schema, imag rollout, environment variables (APP_URL, EMAIL_SENDER, ACS_CONNECTION_STRING), removal of the old shared-password se creating the first account link, and merging to main. Integration tests are re-run on the test schema before resuming.